Prof. Dr. Gogol-Döring

Theoretische Informatik 1

Klausur Übungsaufgaben

▸ Aufgabe 1: Übergangsfunktionen von endlichen Automaten

Eine mathematische Funktion bildet die Elemente einer Definitionsmenge auf Elemente einer Wertemenge ab. Sei Σ das endliche Alphabet und Q die endliche Zustandsmenge eines endlichen Automaten; $s \in Q$ sei der Startzustand, $F \subseteq Q$ die Menge der akzeptierenden Endzustände.

1. Welche Art von Funktion ist die Übergangsfunktion δ eines *DEA*?
2. Welche Art von Funktion ist die Übergangsfunktion δ eines *NEA* (ohne ε-Übergänge)?
3. Welche Art von Funktion ist die Übergangsfunktion δ eines *ε-NEA*?

Geben Sie jeweils die Definitionsmenge und die Wertemenge von δ an.

▸ Aufgabe 2: Sprache ⇒ DEA

Sei Σ = {"A", "O"} ein Alphabet mit zwei Zeichen und Σ* die Menge aller Strings mit Zeichen aus Σ. Zeichnen Sie für jede der folgenden Sprachen $L \subseteq \Sigma^*$ einen *deterministischen* endlichen Automaten, der die Sprache L akzeptiert. Ihr Automat soll also alle Strings akzeptieren, die in der Sprache sind, dabei aber keinen String akzeptieren, der nicht in der Sprache ist.

1. L = {w ∈ Σ* | in w folgt niemals ein "O" auf ein "A"}
2. L = {w ∈ Σ*| w enthält sowohl den String "AA" als auch den String "OO"}
3. L = {w ∈ Σ*| w enthält den Buchstaben "A" entweder genau zweimal oder genau dreimal}

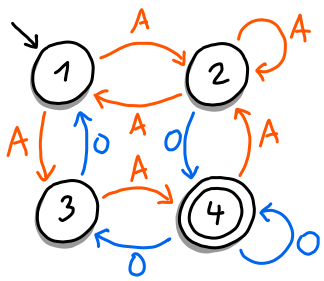
▸ Aufgabe 3: Sprache ⇒ Automaten und reguläre Ausdrücke

Zeichnen Sie zu jeder der folgenden Sprachen L einen *endlichen Automaten* (egal ob DEA, NEA oder ε-NEA), der L akzeptiert, und geben Sie jeweils einen *regulären Ausdruck* an, der L beschreibt

1. L = {w ∈ {A, O}* | w enthält genau ein "A" oder genau ein "O"}
2. L = {w ∈ {A, O}* | entweder enthält w genau ein "A" oder genau ein "O"}
3. L = {w ∈ {A, O}* | w enthält nicht den Teilstring "OAO"}

▸ Aufgabe 4: Pfade im NEA

Betrachten Sie folgenden nichtdeterministischen endlichen Automaten:



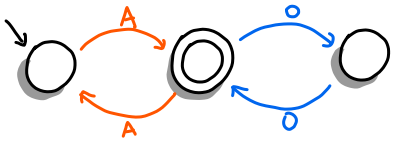
Geben Sie zu jedem der folgenden Strings eine Folge $q_1 \rightarrow q_2 \rightarrow q_3 \rightarrow \ldots \rightarrow q_n$ von Zuständen an, die der Automat durchlaufen kann, um den String zu akzeptieren.

1. AOAO
2. AAAAOAA
3. AAOOOA

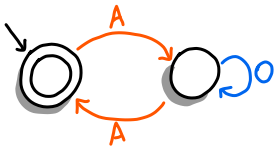
▸ Aufgabe 5: Endlicher Automat ⇒ Sprache

Geben Sie zu jedem der folgenden endlichen Automaten die Sprache an, die der Automat akzeptiert. Dabei dürfen Sie die Sprache in der Form "L = {…}" definieren oder einen regulären Ausdruck angeben, der die Sprache beschreibt.

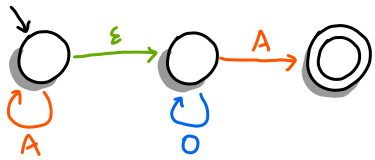
1.



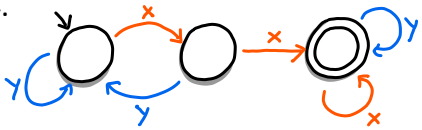
2.



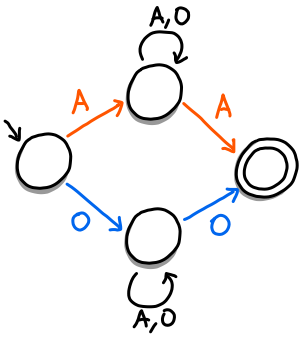
3.



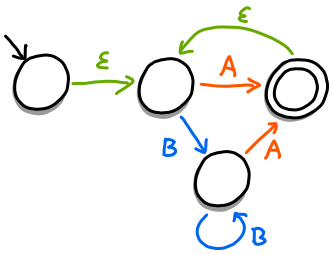
4.



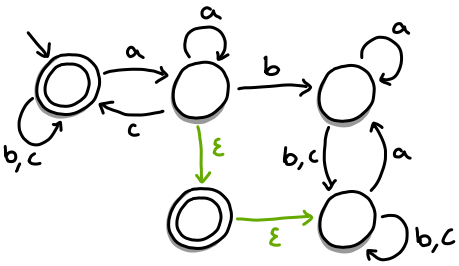
5.



6.



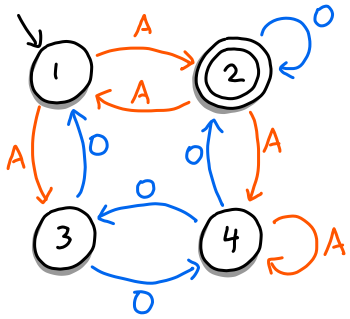
7.



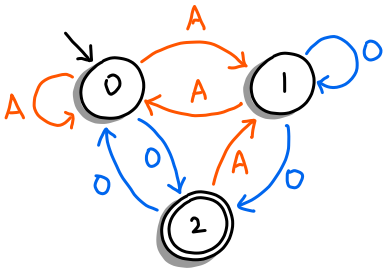
▸ Aufgabe 6: NEA in DEA verwandeln

Wandeln Sie jeden der folgenden NEA in einen DEA um. Markieren Sie eindeutig den Startzustand und alle akzeptierenden Endzustände Ihres DEA. Kennzeichnen Sie jeden Zustand Ihres DEA mit der ihm entsprechenden Teilmenge der erreichbaren Zustände im NEA. Dabei müssen verschiedene Zustände des DEA mit verschiedenen Teilmengen beschriftet werden.

1.



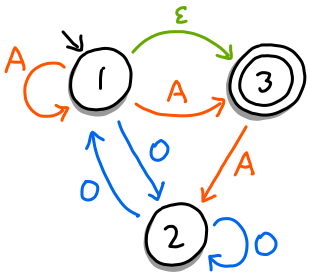
2.



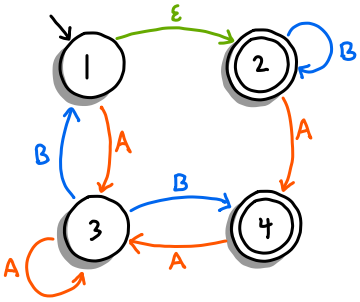
▸ Aufgabe 7: ε-NEA in DEA verwandeln

Wandeln Sie folgenden ε-NEA in einen DEA um. Markieren Sie eindeutig den Startzustand und alle akzeptierenden Endzustände Ihres DEA. Kennzeichnen Sie jeden Zustand Ihres DEA mit einer Teilmenge von {1, 2, 3}, die der ihm entsprechenden Teilmenge der jeweils erreichbaren Zustände des ε-NEA entspricht. Ihr DEA darf nicht mehr als 7 Zustände haben.

1.



2.



▸ Aufgabe 8: Strings in der Sprache eines regulären Ausdrucks

Betrachten Sie folgenden regulären Ausdruck:

$$(ABB^*|ABA^*AA)^*B$$

Entscheiden Sie zu jedem der folgenden Strings, ob er in der Sprache ist, die von diesem regulären Ausdruck beschrieben wird:

1. ε
2. ABAAAAABBB
3. ABABAAB
4. ABABABAABB
5. ABAAABBABB

▸ Aufgabe 9: Sprachen ⇒ Regulärer Ausdruck

Sei Σ = {"A", "O"} ein Alphabet mit zwei Zeichen und Σ* die Menge aller Strings mit Zeichen aus Σ. Geben Sie für jede der folgenden Sprachen L ⊆ Σ* einen regulären Ausdruck an, der diese Sprache definiert:

1. L = {w ∈ Σ* | der vorletzte Buchstabe von w ist ein "A"}
2. L = {w ∈ Σ* | w enthält "AA" als Teilstring}
3. L = {w ∈ Σ* | die beiden Buchstaben am Anfang und am Ende von w sind unterschiedlich}
4. L = {w ∈ Σ* | in w steht vor oder nach jedem O ein weiteres O}
5. L = {w ∈ Σ* | w enthält eine gerade Anzahl an "A"}

(Aufgabe 10 wurde gelöscht)

▸ Aufgabe 11: Regulärer Ausdruck ⇒ Automat

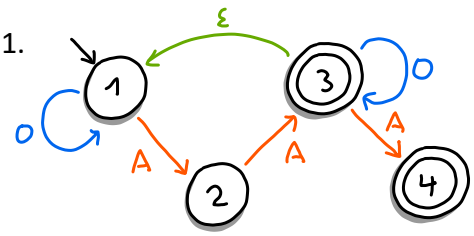
Zeichnen Sie zu jedem der folgenden regulären Ausdrücke einen endlichen Automaten (ε-NEA), der genau die von dem regulären Ausdruck definierte Sprache akzeptiert:

1. Ø|A|O
2. (A*O)*
3. (AO|O*A)*

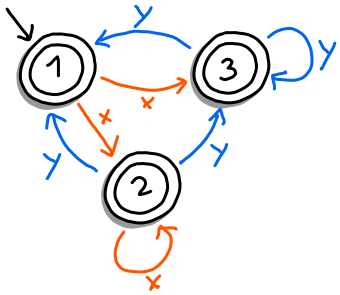
▸ Aufgabe 12: Automat ⇒ Regulärer Ausdruck

Verwandeln Sie folgende endlichen Automaten in reguläre Ausdrücke, die genau die vom jeweiligen Automaten akzeptierte Sprache beschreiben. Verwenden Sie dabei die im Video behandelte Methode, bei der Schritt für Schritt alle Zustände außer dem Start- und dem Endzustand eliminiert werden. Zeichnen Sie, wie der Automat nach jedem Schritt aussieht. Geben Sie am Ende den regulären Ausdruck an.

1.

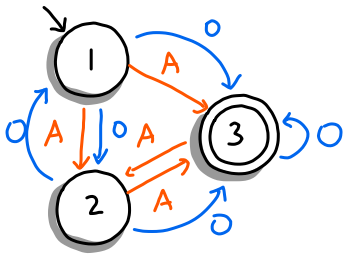


2.



▸ Aufgabe 13: Pumping Lemma

Betrachten Sie folgenden endlichen Automaten:



Geben Sie zu jedem der folgenden Strings w eine Zerlegung w = abc mit b ≠ ε an, so dass gilt: Der Automat akzeptiert alle Strings der Sprache ab*c.

1. w = AAOA
2. w = AOA
3. w = AO

▸ Aufgabe 14: DEA mit mehreren Endzuständen

Bei der Definition von deterministischen endlichen Automaten (DEA) wird mit N ⊆ Q eine Menge akzeptierender Endzustände festgelegt (Q ist die Zustandsmenge), d.h. es kann mehrere akzeptierende Endzustände geben.

1. Geben Sie ein Beispiel für eine reguläre Sprache an, die von keinem DEA akzeptiert werden kann, der nur über einen einzelnen Endzustand verfügt. Definieren Sie diese Sprache durch einen regulären Ausdruck.

2. Anders ist das bei nichtdeterministischen endlichen Automaten mit ε-Übergängen (ε-NEA), denn bei ihnen könnte man grundsätzlich mit nur einem Endzustand auskommen. Begründen Sie, warum es für jede reguläre Sprache L einen ε-NEA mit genau einem Endzustand gibt, der L akzeptiert.

▸ Aufgabe 15: Ableitungsbäume

Betrachten Sie folgende CFG:

S → aX | SXb
X → ba | Sa

Beweisen Sie zu jedem der folgenden Strings, dass er von dieser Grammatik erzeugt werden kann, indem Sie für den String einen Ableitungsbaum zeichnen:

1. aba
2. aaabaaa
3. aababababab

▸ Aufgabe 16: CFG ⇒ Sprache

Geben Sie zu jeder der folgenden kontextfreien Grammatiken die Sprache an, die von ihr erzeugt wird. Definieren Sie die Sprache in der Form "L = {…}" oder − falls das möglich ist − mit einem regulären Ausdruck.

1. S → ε | aS | Sbb
2. S → b | SaS
3. S → bX | Sa
   X → ε | Xb

▸ Aufgabe 17: Sprache ⇒ CFG

Beschreiben Sie zu jeder der folgenden Sprachen eine kontextfreie Grammatik, die diese Sprache erzeugt. Dabei genügt es, alle Produktionsregeln anzugeben. Verwenden Sie Großbuchstaben für Nichtterminale; das Startsymbol sei S.

1. L = {w ∈ {a, b}* | w beginnt und endet mit a}
2. L = {$a^n b a^m$ | n, m ∈ $\mathbb{N}_0$ und n ≥ m}
3. (ab|b|c)*

▸ Aufgabe 18: Backus-Naur-Form

Ein aussagenlogischer Ausdruck bestehe aus einer Reihe von Variablen, die auf die übliche Weise mit den Operatoren *and*, *or* und *not* verknüpft werden können. Außerdem darf man in aussagenlogischen Ausdrücken auch Klammern (also "(" und ")") verwenden − muss das aber nicht. Angenommen, wir haben bereits für die Erzeugung von Variablennamen eine kontextfreie Grammatik (CFG) mit dem Nichtterminal ⟨Variable⟩ durch eine Backus-Naur-Form definiert. Definieren Sie nun auch die restliche Syntax aussagenlogischer Ausdrücke in Backus-Naur-Form. Verwenden Sie ⟨Ausruck⟩ als Startsymbol. Achten Sie darauf, dass ausschließlich wohlgeformte aussagenlogische Ausdrücke von der CFG erzeugt werden.

▸ Aufgabe 19: CFG mit nur einem Nichtterminal

Betrachten Sie folgende kontextfreie Grammatik:

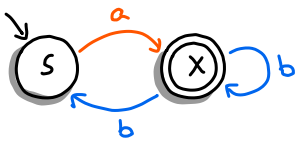
S → a | aXS | YaX | Xb | ZY
X → ScS | ZaX| bb
Y → cZ | YSc | ZYa
Z → XbY | XcZ

Beschreiben Sie eine kontextfreie Grammatik, die genau dieselbe Sprach erzeugt, dabei aber nur ein Nichtterminal verwendet (nämlich das Startsymbol S).

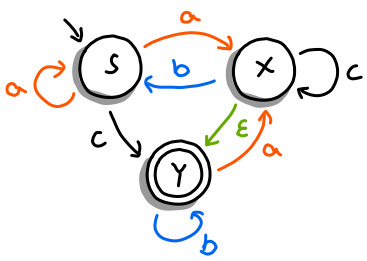
▸ Aufgabe 20: Automat ⇒ CFG

Beschreiben Sie zu jedem der folgenden Automaten eine kontextfreie Grammatik, die genau die Sprache beschreibt, die der Automat akzeptiert. Das Startsymbol der kontextfreien Grammatik sei S.

1.



2.



▸ Aufgabe 21: CFG ⇒ Kellerautomat

Zeichnen Sie zu jeder der folgenden kontextfreien Grammatiken einen Kellerautomaten, der genau die Sprache akzeptiert, die die kontextfreie Grammatik erzeugt.

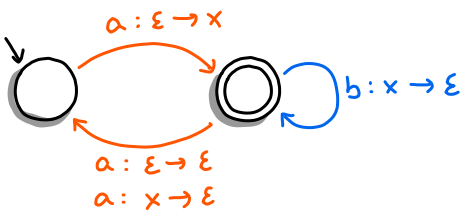
1. S → aSb | bSa | ε

2. S → aSa | bbS | X
   X → Xb | SS | ε

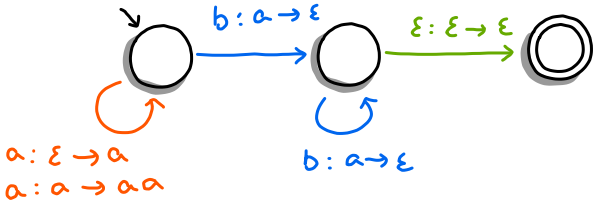
▸ Aufgabe 22: Kellerautomat ⇒ Sprache

Geben Sie zu jedem der folgenden Kellerautomaten die Sprache an, die der Kellerautomat akzeptiert. Definieren Sie dabei die Sprache in der Form "L = {…}" oder – falls das möglich ist – mit einem regulären Ausdruck.

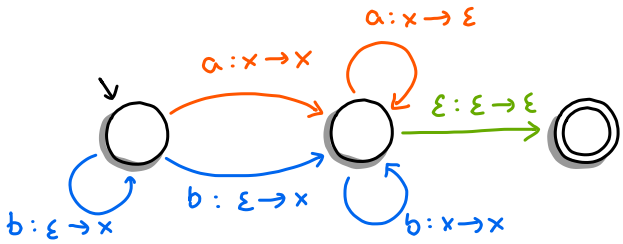
1.



2.



3.



▸ Aufgabe 23: Pumping-Lemma für CFG

Betrachten Sie folgende kontextfreie Grammatik mit Startsymbol S:

S → abSX | Xa
X → Xaa | bS | b

Sei L die Sprache, die von dieser kontextfreien Grammatik erzeugt wird.
Geben Sie einen String w ∈ L an sowie eine Zerlegung w = xuyvz für x, u, y, c, z ∈ {a, b}* mit u ≠ ε oder v ≠ ε, so dass gilt:
$xu^nyv^nz \in L$ für alle $n \in \mathbb{N}_0$.

▸ Aufgabe 24: Loop-Programme verstehen

Jedes der folgenden extended-Loop-Programme erhält in der Variablen "in" einen als Zahl kodierten String mit Buchstaben aus einem Alphabert Σ. Definieren Sie jeweils die Sprache, die das Programm akzeptiert:

1.

```
n ← len(in)
if n % 2
   return true
else
   if a[0] = "A"
      return true
   end
   return false
end
```

2.

```
a ← 0
b ← 0
c ← 0
if len(in) = 0
    return true
end
for i ← 0, …, len(in)−1
    if in[i] = "a"
        ++a
    end
    if in[i] = "b"
        ++b
    end
    if in[i] = "c"
        ++c
    end
end
return (a = b) and (b = c)
```

3.

```
n ← len(in)
i ← 0
loop n
    if in[i] = "A"
        return true
    else
        return false
    end
    ++i
end
return false
```

▸ Aufgabe 25: Sprache ⇒ Loop-Programm

Schreiben Sie zu jeder der folgenden Sprachen ein (extended-) Loop-Programm, das diese Sprache akzeptiert:

1. $L = \{w \in \{a, b, c\}^* \mid$ in w kommen niemals zwei gleiche Buchstaben direkt hintereinander$\}$
2. $L = \{w \in \Sigma^* \mid$ die Länge von w ist eine Primzahl$\}$
3. $L = \{w \in \{a, b\}^* \mid$ Es gibt ein $n \in \mathbb{N}$ so dass $w = u_1u_2…u_n$ mit $u_i = a^ib^i\}$

▸ Aufgabe 26: While-Programm verstehen

Das folgende (extended-) While-Programm bekommt eine Zahl n als Eingabe:

```
k ← 0
while n
    n ← n · (n−5)
    −−n
    ++k
end
return k
```

Was berechnet dieses Programm? Geben Sie eine Definition für Ausgabe des Programms in Abhängigkeit von der Eingabe $\in \mathbb{N}_0$ an! Aus der Definition soll hervorgehen, wann das Programm anhält und wann nicht; wenn das Programm anhält, dann soll die Definition den Rückgabewert benennen.

▸ Aufgabe 27: Suche in Loop-Listen

Schreiben Sie ein Loop-Programm, das als Eingabe eine Liste a von Zahlen sowie eine weitere Zahl x bekommt, und das prüft, ob x die Summe zweier Zahl in a ist! Die Liste sei in einer einzelnen Zahl kodiert; mit count(a) lässt sich die Länge der Liste a (d.h. die Anzahl der in ihr gespeicherten Zahlen) ermitteln; a_i sei das i-te Element der Liste; wobei $i \in \{0, \ldots, \text{count}(a)\}$. Ihr Programm soll genau dann *true* zurückgeben, wenn $i, j \in \{0, \ldots, \text{count}(a)-1\}$ mit $i \neq j$ gibt, so dass $x = a_i + a_j$; andernfalls soll das Programm *false* zurückgeben.

▸ Aufgabe 28: Goto ⇒ While

Betrachten Sie folgendes Goto-Programm:

```
L1:   if n > 10
        n ← n − 3
        goto L2
      else
        n ← n / 2
L2:     if n < 3
          n ← n + 2
        else
          goto L1
        end
      end
      return n
```

Wandeln Sie dieses Programm in ein While-Programm um! Beachten Sie dabei, dass die Programmiersprache While nicht den Befehl *goto* unterstützt! Ihr While-Programm soll für jede Eingabe $n \in \mathbb{N}_0$ dasselbe berechnen wie das hier gezeigte Goto-Programm. Wenn das Goto-Programm für ein $n \in \mathbb{N}_0$ nicht anhält, soll Ihr While-Programm ebenfalls für dieses $n \in \mathbb{N}_0$ nicht anhalten.

▸ Aufgabe 29: Reterna verstehen

Geben Sie zu jeder der folgenden Reterna-Funktionen an, welchen Wert sie zurückgeben, wenn sie mit dem Argument n = 7 aufgerufen werden.

1. `func1(n) = n > 1 ? func1(n−2) + 1: 0`
2. `func2(n) = n > 1 ? func2(n−1) · func(n−2) : n+1`
3. `func3(n) = n ≥ 10 ? n+1 : func3(func3(n+1))`

▸ Aufgabe 30: Reterna ⇒ Sprache

Sei Σ* ein Alphabet; "A" ∈ Σ. Die folgenden Reterna-Funktionen **language1**, **language2** und **language3** bekommen als Eingabe jeweils einen als Zahl kodierten String a ∈ Σ* übergeben. Geben Sie zu jeder dieser Reterna-Funktionen die Sprache an, die sie akzeptiert! Dabei dürfen Sie die Sprache in der Form "L = {…}" definieren oder einen regulären Ausdruck angeben, der die Sprache beschreibt.

1. **language1**(a) = len(a) % 2 ? a[len(a)/2] = "A" : *false*

2. **language2**(a) = len(a) % 2 ? *false* : L2(a, len(a)−1)
   L2(a, i) = (a[i] = "A") *and* (i ? L2(a, i−2) : *true*)

3. **language3**(a) = L3(a, len(a))
   L3(a, n) = n % 3 ? *false* : match(a, n/3, 0, n/3) *and* match(a, n/3, n/3, 2·n/3)
   match(a, m, i, j) = m ? (a[i] = a[j]) *and* match(a, m−1, i+1, j+1) : *true*

▸ Aufgabe 31: Sprache ⇒ Reterna

Sie Σ = {A, B, C} ein Alphabet. Definieren Sie für jede der folgenden Sprachen L ⊆ Σ* ein (extended) Reterna-Programm, das genau die Sprache L akzeptiert! Die Funktionen bekommen jeweils als Eingabe einen als Zahl kodierten String a und soll genau dann *true* zurückgeben, wenn a ∈ L, ansonsten *false*.

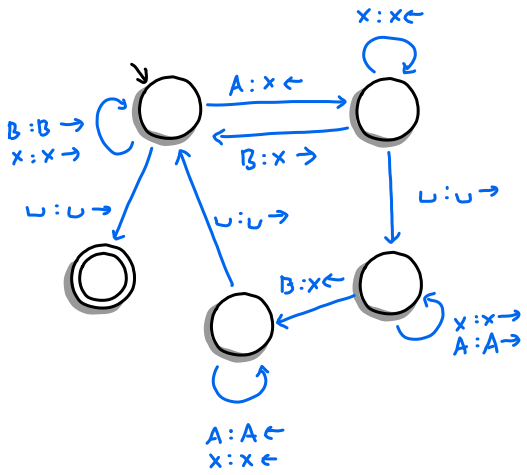
1. L = {w ∈ Σ* | w enthält den Buchstaben A}
   = Σ*AΣ*
2. L = {w ∈ Σ* | w enthält mindestens so viele A wie B}
3. L = {w ∈ Σ* | wenn in w der Buchstabe A vorkommt, kommen rechts davon in w keine B vor}
   = (B|C)*|Σ*A(A|C)*

▸ Aufgabe 32: Faktorisieren mit Reterna

Programmieren Sie eine Funktion **largestDivisor**(n), die für alle n ∈ ℕ mit n ≥ 2 den größten Teiler von n berechnet, der kleiner als n ist!

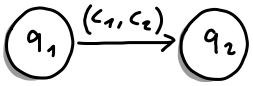
▸ Aufgabe 33: Turingmaschine verstehen

Sei Σ = {A, B} das Eingabealphabet und Γ = {A, B, X, ␣} das Bandalphabet. Geben Sie die Sprache L ⊆ Σ* an, die die folgende Turingmaschine akzeptiert! Definieren Sie die Sprache in der Form "L = {…}"!



▸ Aufgabe 34: Zweikopf-Automaten

Ein "endlicher Automat mit zwei Leseköpfen" (2KEA) ist einem ε-NEA (nichtdeterministischer endlicher Automat mit ε-Übergängen) ähnlich, doch anders als dieser liest er den Eingabestring mit zwei unabhängig voneinander arbeitenden Leseköpfen. Beide Leseköpfe arbeiten auf demselben Eingabestring, und wenn der 2KEA startet, zeigen beide Leseköpfe auf das erste Zeichen des Eingabestrings. Die Übergangsfunktion eines 2KEA ist eine Funktion $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$, die wie folgt interpretiert wird: Wenn $q_2 \in \delta(q_1, c_1, c_2)$, dann darf der 2KEA vom Zustand q_1 in den Zustand q_2 überwechseln, wenn er dabei mit Lesekopf 1 das Zeichen c_1 und mit Lesekopf 2 das Zeichen c_2 liest. (Dabei ist Q die Zustandsmenge und Σ das Eingabealphabet.) Als Zeichnung können wir das darstellen, indem wir an den Übergang von q_1 nach q_2 das Paar (c_1, c_2) schreiben:



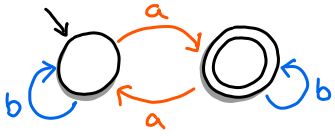
Wenn eines der Zeichen c_1 oder c_2 gleich ε ist, so bedeutet dies, dass der betreffende Lesekopf kein Zeichen lesen soll, sondern auf seiner aktuellen Position verbleibt. Auf diese Weise kann man erreichen, dass die beiden Leseköpfe an verschiedenen Positionen im Eingabestring stehen. Sind c_1 und c_2 gleichzeitig ε, dann ist ein Übergang von q_1 nach q_2 möglich, ohne dass dabei ein Zeichen gelesen wird. Ein 2KEA akzeptiert einen Eingabestring genau dann, wenn es eine Folge von Zuständen $q_1 \to q_2 \to \ldots \to q_n$ vom Startzustand $s = q_1$ zu einem akzeptierenden Endzustand $q_n \in N$ gibt, bei dem jeder Übergang $q_i \to q_{i+1}$ legal nach einer Übergangsregel ausgeführt wird, und am Ende beide Leseköpfe den kompletten Eingabestring gelesen haben. Dabei ist N die Menge der akzeptierenden Endzustände; in der Zeichnung werden diese Zustände mit doppeltem Rand dargestellt.

Zeichnen Sie einen 2KEA, der die Sprache $L = \{a^n b a^n \mid n \in \mathbb{N}_0\}$ akzeptiert.

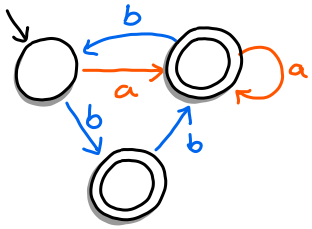
▸ Aufgabe 35: Zwei DEA schneiden

Betrachten Sie die folgenden deterministischen endlichen Automaten (DEA) A_1 und A_2:

A_1



A_2



Der Automat A_1 akzeptiere die Sprache L_1 und der Automat A_2 die Sprache L_2.

Zeichnen Sie einen DEA, der die Sprache $L_1 \cap L_2$ akzeptiert. Ihr Automat soll also genau dann einen String akzeptieren, wenn er von beiden Automaten A_1 und A_2 akzeptiert wird.
Achtung: Ihr Automat soll deterministisch sein!